



Team 2

Yersultan Doszhan

Alexandra Suchshenko

Nursultan Soodonbekov

RuMMSD: Comparative Study on Russian and English Multimodal Sarcasm Detection

Final Project Presentation

Table of contents

Overview

1

4

Results

Dataset

2

5

Findings

Model training

3

6

Limitations & Future work

Overview

Gap

There is a need to advance Russian sarcasm detection beyond simple baseline models. While previous works like RuSentiment (Rogers et al., COLING 2018) relied on basic classifiers and purely textual analysis, sarcasm often manifests through the interplay of text, images, and formatting - particularly in social media contexts.

Aim

- Create a comprehensive Russian social media sarcasm dataset, incorporating text and image modalities
- Evaluate how different modalities and their interactions contribute to sarcasm detection accuracy in Russian social media content
- Establish new state-of-the-art benchmarks for Russian multimodal sarcasm detection that better capture the nuanced ways sarcasm is expressed online

Dataset (Russian) updates

1. Dataset Expansion & Refinement
 - a. Doubled data samples: 198 samples → **398 samples**
 - b. Reduced temporal bias: Data collection spans over four years (**Jan 2020** to Nov 2024)
 - c. Diversified samples:
 - i. Not relying solely on hashtags
 - ii. **50% sampled from individual profiles**
 - iii. 50% from top 10 irony/comedy-related public groups
 - d. Reduced image bias: Made sure **images do not contain group logos**
2. Integration of Image Caption Modality (for Multimodal CLIP and Claude models)

Dataset (Russian): Expansion & Refinement

1. Initial data collection



Targethunter.ru and **VK.barkov.net**:

Services for scraping target audience data, posts, and comments.



Scraping guidelines

Russian posts that contain:

1. Images
2. For positive examples (sarcastic) → Posts with specific hashtags (e.g., #sarcasm, #humor, #jokes)
#сарказм, #юмор, #шутки

For negative examples (non-sarcastic) → Posts without sarcasm-related hashtags

Stop-words: sarcasm, sarcastic, irony, ironic, humor, jokes

(сарказм, саркастично, ирония, иронично, юмор, шутки)

Period (before): January 1, 2023 - November 5, 2024

Period (added): January 1, 2020 - December 1, 2022

Total posts extracted: > 10000

Dataset (Russian): Expansion & Refinement

2. Data Cleaning

3. Text Preprocessing

Normalization and
Tokenization



Current RuMMSD Composition

Total posts after manual agreement filtering:

Sarcastic posts: 199

Non-sarcastic posts: 199

manual agreement filtering (label=1)

[1, 1, 1]	# all raters agree	→ Keep sample
[0, 1, 1]	# rater 1 disagrees	→ Keep sample
[0, 0, 1]	# rater 1 & 2 disagree	→ Remove sample
[0, 0, 0]	# all raters agree	→ Remove sample

4. Dataset Split (**70% train set, 30% test set**)

Dataset (Russian): Data annotation

Current RuMMSD Composition

Sarcastic posts: 199

Non-sarcastic posts: 199

Examples

(positive)



“Полный **лакшери супер курорт** 🍷🍸🍹”

“Total **luxury super resort** 🍷🍸🍹”

(negative)



“**Люблю** запах **хорошего крепкого кофе** и корицы...”

“I **love** the smell of **good strong coffee** and cinnamon...”

Model selection

Selected models (before): GPT-4 Turbo, Claude-3-haiku, LLaMA 3-8B

Selected models (after): GPT-4o, GPT-4o-mini, Claude-3-haiku, Multimodal CLIP

Why?

- GPT-4o and GPT-4o-mini surpasses GPT-3.5 Turbo and other small models on academic benchmarks across both textual intelligence and multimodal reasoning
- Cost-effective handling of non-English text
- CLIP: For image caption modality generation

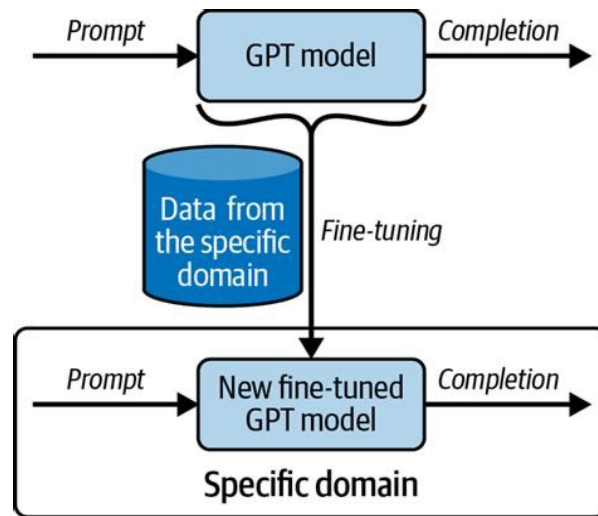
Model training: Fine-tuning

Model Architecture and Setup:

- A generative language model with a transformer-based architecture.
- Equipped with pre-trained embeddings and attention layers trained on vast amounts of text data.
-

Training Process:

- Paired image-text data in batches:
 - Images → Vision encoder
 - Text (tokenized) → Tokenized → Pre-trained GPT-4o
- Image embeddings → Classifier → Final predictions
- Loss Calculation → Cross-Entropy Loss
- Optimization → Backpropagation to update all parameters



Model training: GPT-4o

GPT-4o

Supports text and vision in the API

1. Model input:
 - a. Text only
 - b. Image only
 - c. Multimodal (Text + Image)
2. Model settings/parameters:

☞ Trained tokens	335,680
🔄 Epochs	10
☰ Batch size	16
📖 LR multiplier	1.28
🌱 Seed	22

```
# Function to query the fine-tuned model with text and image
def query_fine_tuned_model(text, image_url):
    print(f"Querying model with:\nText: {text}\nImage URL: {image_url}")
    try:
        response = openai.ChatCompletion.create(
            model=fine_tuned_model_id,
            messages=[
                {
                    "role": "system",
                    "content": "You are a helpful assistant that answers with only '1' for sarcastic "
                               "or '0' for not sarcastic when analyzing the text and image."
                },
                {
                    "role": "user",
                    "content": f"Does this seem sarcastic?\nText: {text}\nImage URL: {image_url}"
                }
            ],
            max_tokens=2
        )
        # Extract the model's prediction
        model_output = response.choices[0].message["content"].strip()
        return int(model_output)
```

Prompting (Multimodal)

Model training: GPT-4o

GPT-4o

Supports text and vision in the API

Results:

Modality	Class	Precision	Recall	F1-Score	Accuracy
Text	Non-sarcastic	0.76	0.89	0.82	0.81
	Sarcastic	0.88	0.74	0.81	
Image	Non-sarcastic	0.49	0.93	0.64	0.50
	Sarcastic	0.67	0.13	0.21	
Multimodal (Text + Image)	Non-sarcastic	0.89	0.91	0.90	0.91
	Sarcastic	0.92	0.91	0.91	

* Best performance shown in **bold**

(Tested on 119 samples)

Model training: GPT-4o-mini

GPT-4o-mini**Supports text and vision in the API**

1. Model input:

- Text only
- Image only
- Multimodal (Text + Image)
- Output: `ft:gpt-4o-mini-2024-07-18`

2. Model settings/parameters:

⚙️ Trained tokens	67,136
🔄 Epochs	2
☰ Batch size	16
📊 LR multiplier	1.28
🎲 Seed	22

```
# Function to query the fine-tuned model with text and image
def query_fine_tuned_model(text, image_url):
    print(f"Querying model with:\nText: {text}\nImage URL: {image_url}")
    try:
        response = openai.ChatCompletion.create(
            model=fine_tuned_model_id,
            messages=[
                {
                    "role": "system",
                    "content": "You are a helpful assistant that answers with only '1' for sarcastic "
                               "or '0' for not sarcastic when analyzing the text and image."
                },
                {
                    "role": "user",
                    "content": f"Does this seem sarcastic?\nText: {text}\nImage URL: {image_url}"
                }
            ],
            max_tokens=2
        )
        # Extract the model's prediction
        model_output = response.choices[0].message["content"].strip()
        return int(model_output)
```

Prompting (Multimodal)

Model training: GPT-4o-mini

GPT-4o-mini

Supports text and vision in the API

Results:

Modality	Class	Precision	Recall	F1-Score	Accuracy
Text	Non-sarcastic	0.65	0.95	0.77	0.73
	Sarcastic	0.92	0.54	0.68	
Image	Non-sarcastic	0.49	0.52	0.50	0.52
	Sarcastic	0.55	0.52	0.54	
Multimodal (Text + Image)	Non-sarcastic	0.80	0.93	0.86	0.85
	Sarcastic	0.92	0.80	0.85	

* Best performance shown in **bold**

(Tested on 119 samples)

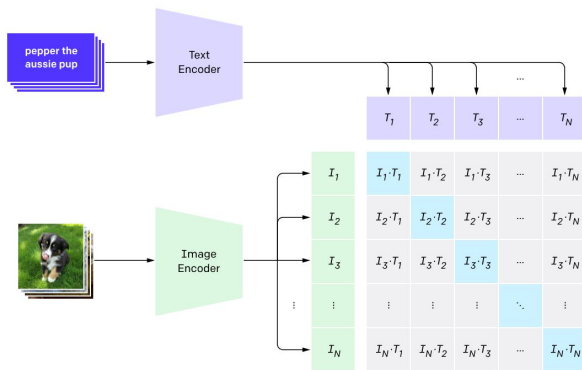
Model training: Multimodal CLIP

CLIP

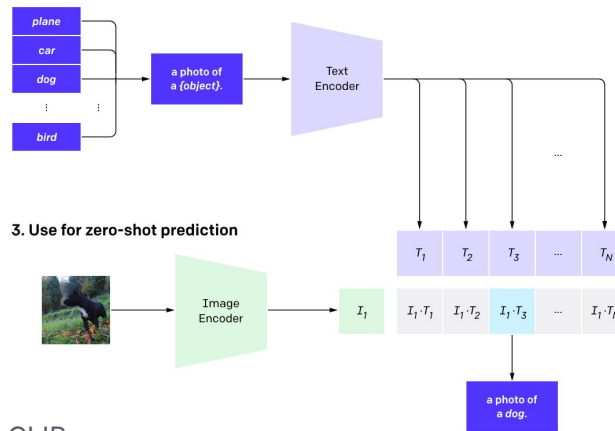
Contrastive Language–Image Pre-training

- Learns from unfiltered, highly varied, and highly noisy data, and is intended to be used in a zero-shot manner
- More flexible and general than ImageNet models

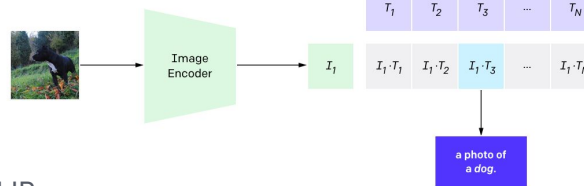
1. Contrastive pre-training



2. Create dataset classifier from label text



3. Use for zero-shot prediction



Overview of CLIP

Model training: Multimodal CLIP (Cont.)

CLIP

Contrastive Language–Image Pre-training

1. Model input:
 - a. Text only
 - b. Image only
 - c. Multimodal (Text + Image)

```
import torch.nn as nn

class MultimodalClassifier(nn.Module):
    def __init__(self, projection_dim, hidden_dim, num_classes):
        super().__init__()
        self.fc1 = nn.Linear(projection_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3)
        self.fc2 = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x

classifier = MultimodalClassifier(model.config.projection_dim, 512, 2)
classifier.to(device)
```

```
# Fine-tuning the entire model
for param in model.parameters():
    param.requires_grad = True

optimizer = AdamW([
    {'params': model.parameters(), 'lr': 5e-6}, # Lower LR for CLIP backbone
    {'params': classifier.parameters(), 'lr': 1e-4} # Higher LR for classifier head
])

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=10)

for epoch in range(2): # Train for more epochs
    model.train()
    classifier.train()
    total_loss = 0

    for batch in train_dataloader:
        inputs = processor([
            text=batch["text"],
            images=[Image.open(p) for p in batch["image_path"]],
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=77
        ])

        input_ids = inputs["input_ids"].to(device)
        attention_mask = inputs["attention_mask"].to(device)
        pixel_values = inputs["pixel_values"].to(device)
        labels = batch["label"].to(device)

        optimizer.zero_grad()

        # Forward pass
        outputs = model(input_ids=input_ids, attention_mask=attention_mask, pixel_values=pixel_values)
        image_embeddings = outputs.image_embeds
        logits = classifier(image_embeddings)

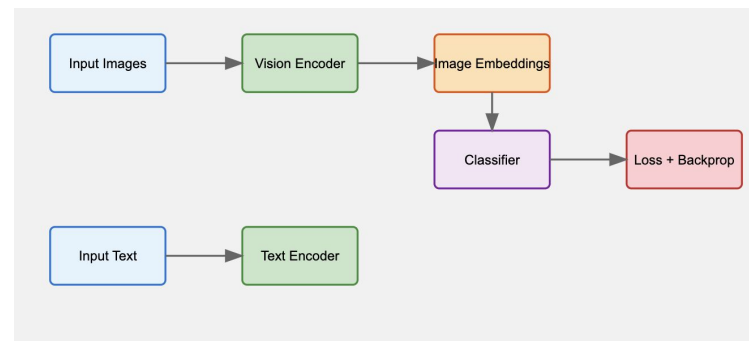
        # Compute loss
        loss = loss_fn(logits, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    scheduler.step()
    print(f"Epoch {epoch + 1}, Loss: {total_loss}")
```

Model training: Multimodal CLIP Fine-tuning

1. Model Architecture and Setup:
 - a. Pre-trained CLIP model with image and text encoders
 - b. A custom classifier is added on top of CLIP's image embeddings
 - c. The entire model is set for fine-tuning (all parameters are trainable)
2. Training Process:
 - a. Paired image-text data in batches:
 - i. Images → Vision encoder
 - ii. Text (tokenized) → Text encoder
 - b. Image embeddings → Classifier → Final predictions
 - c. Loss calculation + Backpropagation
3. Trains for 10 epochs with full model fine-tuning



Model training: Multimodal CLIP

CLIP

Contrastive Language–Image Pre-training

Results:

Modality	Class	Precision	Recall	F1-Score	Accuracy
Text	Non-sarcastic	0.72	0.75	0.73	0.72
	Sarcastic	0.74	0.71	0.72	
Image	Non-sarcastic	0.56	0.57	0.56	0.56
	Sarcastic	0.57	0.56	0.56	
Multimodal (Text + Image)	Non-sarcastic	0.97	0.89	0.93	0.94
	Sarcastic	0.91	0.97	0.95	

* Best performance shown in **bold**

(Tested on 119 samples)

Model training: Claude-3-haiku

- Generating synthetic captions with **BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation**

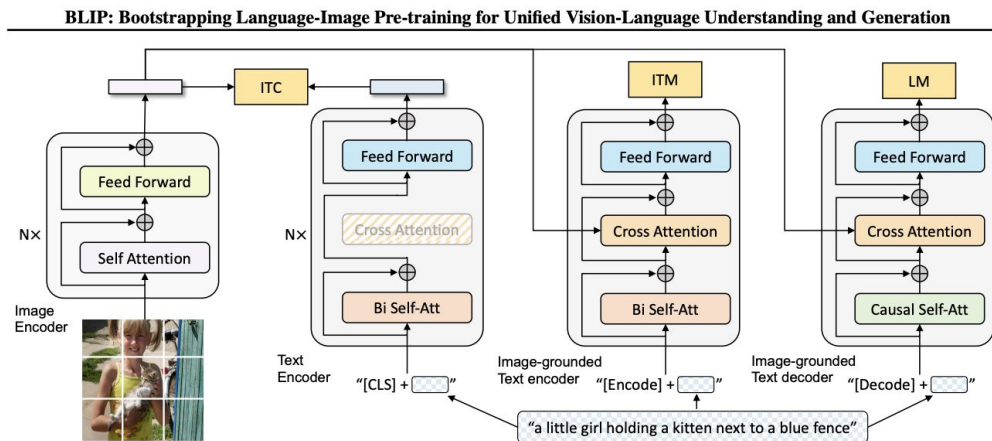


Figure 2. Pre-training model architecture and objectives of BLIP (same parameters have the same color).

Model training: Claude-3-haiku

Claude-3-haiku

1. Model input:
 - a. Text only
 - b. Image only
 - c. Multimodal (Text + Image)
2. Model settings/parameters:
 - a. On the right image

```
# Create the message
messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text":
                "You are a helpful assistant that answers with only '1' for sarcastic"
                "or '0' for not sarcastic when analyzing the text and image."},
            {"type": "text", "text": text},
            {"type": "image", "source": {"type": "base64", "media_type": "image/jpeg",
                                      "data": encoded_image}}
        ]
    }
]

# Define request body
body = {
    "anthropic_version": "bedrock-2023-05-31",
    "max_tokens": 2,
    "messages": messages,
    "temperature": 0.1,
    "top_p": 0.9
}

# Invoke Claude Haiku
response = bedrock_client.invoke_model(
    modelId="anthropic.claude-3-haiku-20240307-v1:0",
    accept="application/json",
    contentType="application/json",
    body=json.dumps(body)
)
```

Prompting (Multimodal)

Model training: Claude-3-haiku

Claude-3-haiku

1. We have used few-shot learning with examples

```
# Prepare the few-shot learning prompt
few_shot_prompt = "You are a helpful assistant trained to analyze text and images together"
"Your goal is to classify the input as '1' for sarcastic or '0' for not sarcastic."
"Only output a single number ('1' or '0') without any additional explanation"
"Use the image description to interpret the text. Here are some examples:\n\n"

for example in few_shot_examples:
    few_shot_prompt += f"Example:\nText: {example['text']}\nImage Description: {example['image_description']}\n\n"
    Label: "{example['label']}"\n\n"

few_shot_prompt += "Now analyze:\n"
```

```
# Define the few-shot examples
few_shot_examples = [
    {
        "text": "Wow, another Monday. So excited.",
        "image_description": "A messy office with scattered papers and an overflowing trash bin.",
        "label": "1"
    },
    {
        "text": "Finally, a quiet weekend.",
        "image_description": "A crowded park with children playing and loud music from speakers.",
        "label": "1"
    },
    {
        "text": "This meal looks delicious!",
        "image_description": "A burnt steak and wilted vegetables on a plate.",
        "label": "1"
    },
    {
        "text": "Weather is so great today!",
        "image_description": "A flooded street with heavy rain and abandoned vehicles.",
        "label": "1"
    },
    {
        "text": "I just got promoted! Yay!",
        "image_description": "A man is smiling while holding a certificate in front of a cheering crowd.",
        "label": "0"
    },
    {
        "text": "Dinner with family. Perfect night.",
        "image_description": "A cozy dining room with candles and neatly arranged plates.",
        "label": "0"
    }
]
```

Model training: Claude-3-haiku

Claude-3-haiku

Results:

Modality	Class	Precision	Recall	F1-Score	Accuracy
Text	Non-sarcastic	0.72	0.79	0.75	0.75
	Sarcastic	0.79	0.71	0.75	
Image	Non-sarcastic	0.53	0.56	0.54	0.54
	Sarcastic	0.56	0.52	0.54	
Multimodal (Text + Image)	Non-sarcastic	0.85	0.83	0.84	0.84
	Sarcastic	0.84	0.85	0.84	

* Best performance shown in **bold**

(Tested on 119 samples)

Results - Model intercomparison

Data & Label

- Dataset: 198 sarcastic, 198 non-sarcastic
- 1 -sarcastic, 0 - non-sarcastic

Comparison

Modality	Method	Precision	Recall	F1-score	Accuracy
Text	GPT-4o	0.88	0.74	0.81	0.81
	GPT-4o-mini	0.92	0.54	0.68	0.73
	Multimodal CLIP	0.74	0.71	0.72	0.79
Image	GPT-4o	0.67	0.13	0.21	0.50
	GPT-4o-mini	0.55	0.52	0.54	0.54
	Multimodal CLIP	0.57	0.56	0.56	0.56
Multimodal	GPT-4o	0.92	0.91	0.91	0.91
	GPT-4o-mini	0.92	0.80	0.85	0.85
	Multimodal CLIP	0.91	0.97	0.95	0.94

* Only for **sarcastic class**

Findings

1. Multimodal superiority:
 - a. CLIP model: 94% accuracy (best performing)
 - b. GPT-4o: 91% accuracy
 - c. GPT-4o-mini: 85% accuracy
 - d. Consistently outperformed single-modality approaches
2. Competitive with English benchmarks:
 - a. Comparable to state-of-the-art English MMSD systems
 - b. Similar patterns in modality effectiveness
 - c. Demonstrated cross-lingual applicability of multimodal approaches

Findings: Comparison with English MMSD

- Comparison with English MMSD Benchmarks
 - Our multimodal approach (90-91%) outperforms most English MMSD benchmarks, including:
 - MMMA (82.6%)
 - Multi-view CLIP (85.6%)
 - Approaches top-tier performance of Retrieval-LLaVA (89.4%)
- Text-only performance (73-81%) is comparable to English benchmarks (75-80%)
- Image-only performance (50-56%) is notably lower than English benchmarks (60-65%)

Paradigm	Model	MMSD			
		Acc	P	R	F1
Multi-Modal PLMs	MMMA [17]	86.7	80.2	84.4	82.6
	ResNet+BERT [49]	84.8	77.8	84.1	80.8
	MILNet [50]	89.5	85.2	89.2	87.1
	Multi-view CLIP [51]	88.3	82.7	88.7	85.6
Multi-Modal LLMs	Retrieval-LLaVA 1.5 [47]	90.0	89.3	89.6	89.4
	Wenxin 4	60.0	52.6	31.5	39.4
	Qwen-VL-Plus	64.4	52.7	95.0	67.8
	GPT-4V	71.0	59.4	96.5	73.5

SarcasmBench

Modality	Method	F1(%)	Macro-F1		
			Pre(%)	Rec(%)	F1(%)
Text	TextCNN [42]	75.32	78.03	78.28	78.15
	SIARN [44]	75.63	80.34	78.81	79.57
	SMSD [45]	75.82	80.87	78.20	79.51
	Bi-LSTM [43]	77.53	80.97	80.13	80.55
	BERT [46]	80.22	81.31	80.87	81.09
Image	ResNet [39]	61.53	60.12	73.08	65.97
	ViT [40]	63.43	65.68	71.35	68.40
	ConvNeXt [41]	63.62	64.85	72.73	68.56
Multimodal	HFM [21]	80.18	79.40	82.45	80.90
	Res-Bert* [9]	80.85	78.87	84.46	81.57
	Att-Bert* [9]	80.90	80.87	85.08	82.92
	InCrossMGs [11]	82.84	85.39	85.80	85.60
	CMGCN* [12]	83.45	–	–	85.61
	KnowleNet*	86.33	88.83	88.21	88.51

KnowleNet

Limitations

Insufficient Dataset: Limited high-quality datasets for multimodal tasks hinder training and model performance.

Dataset Bias: Improper datasets lead to skewed predictions and poor generalization.

Limited Multimodal LLMs: Few LLMs handle multimodal tasks, restricting development and deployment.

Conclusion

First comprehensive Russian multimodal sarcasm detection dataset:

- 398 samples (doubled from initial 198)
- Balanced distribution: 199 sarcastic, 199 non-sarcastic posts
- Temporal diversity: Data spans from Jan 2020 to Nov 2024
- Reduced annotation bias through diverse sampling strategies

Novel cross-lingual insights:

- First comparative study between Russian and English multimodal sarcasm detection
- Demonstrated that multimodal approaches enhance sarcasm detection accuracy in Russian (90-91%) similar to English benchmarks
- Identified language-specific challenges in visual-textual sarcasm detection

Key Insights

- Multimodal fusion significantly improves sarcasm detection accuracy
- Russian language models can achieve comparable or better performance than English counterparts
- Image-only analysis remains challenging across both languages

Thank you!



nursultan@kaist.ac.kr

edoszhan@kaist.ac.kr

alexandra.suchshenko@kaist.ac.kr